

Homework problems (Apni kaksha)

<https://ayushtripathi1025.notion.site/Notes-Module-3-787808df788b4e2690e1d32a4f163b37>

### Structure of a C Program:

A C program is organized into several key components:

- **Preprocessor Directives:** These directives, indicated by the # symbol, include headers, define macros, and perform preprocessing tasks. They are processed before compilation.
- **Global Declarations:** This section includes variable and function declarations that are accessible throughout the program. These declarations appear before the main function.
- **Main Function:** The main function is the program's entry point, where execution begins. It contains local variable declarations, function calls, and control structures (loops, conditionals).
- **Functions:** Besides the main function, additional functions can be defined to perform specific tasks. These can be declared before or after the main function, and their declarations must be included in the global declarations or a separate header file.
- **Comments:** Comments provide notes and explanations within the code. They are ignored by the compiler and can be single-line (starting with //) or multi-line (enclosed between /\* and \*/).

### **Example C Program**



```
#include<stdio.h> // Preprocessor Directive
int global_variable; // Global Declaration
void my_function(); // Function Declaration
int main(){ // Main Function
    Int local_variable; // Local Declaration /* This is a comment
*/
    printf("Hello, World!\n");
    my_function();
    return 0;
}

Void my_function(){ // Function Definition//
    printf("This is my function.\n");
}
```

**Overview:** This program includes:

- A preprocessor directive to include the standard I/O library.
- A global integer variable declaration.
- A function declaration for my\_function().
- The main function with a local variable, a print statement, and a call to my\_function().

### **Debugging Techniques in C**

Debugging is a crucial part of software development, and it applies to C programs as well.

Here are some effective techniques for debugging your C programs:

- 1.
2. **Print Statements:** Insert print statements to display variable values and relevant information at various points in the program. This helps identify where issues arise.
3. **Debuggers:** Utilize built-in debuggers provided by C compilers to step through code line by line, set breakpoints, and inspect variable states and memory.
4. **Trace Logs:** Use trace logs to record the program's behaviour by writing messages to a log file at key points. This aids in tracking down problems and understanding variable values.
5. **Assertions:** Implement assertions to check conditions and abort the program if conditions are false. This helps catch errors early and validates assumptions.
6. **Code Reviews:** Have another person review your code to identify errors and logic issues. Fresh perspectives can help catch mistakes you might overlook.

7. **Unit Testing:** Write small tests for individual functions or code sections. This isolates errors and ensures changes in one area don't break others.  
Combining these techniques can significantly enhance the quality of your C programs and help identify bugs early in development.

### Data Types and Sizes in C

C is a strongly-typed language that supports various data types. Here are the commonly used data types and their sizes:

1. **char:** Represents a single character (usually 1 byte).
2. **int:** Represents a signed integer (typically 2 or 4 bytes).
3. **float:** Represents a single-precision floating-point number (usually 4 bytes).
4. **double:** Represents a double-precision floating-point number (typically 8 bytes).
5. **void:** Indicates the absence of type, often used for functions that don't return a value.

#### Modifiers:

- **short:** Specifies a short integer (typically 2 bytes).
- **long:** Specifies a long integer (typically 4 bytes).
- **unsigned:** Indicates an integer that can only take non-negative values.
- **signed:** Indicates an integer that can take both positive and negative values.

C also supports typedef, which allows you to create custom data types based on existing ones, useful for defining custom data structures or more descriptive names.

### Data Type Sizes in C

Be aware that the sizes of data types can vary based on the compiler, operating system, and hardware architecture. It's important to consider platform-specific differences when working with data types in C.

### Modifiers in C

Modifiers are keywords that adjust the behavior of variables or data types. Here are some commonly used modifiers:

1. **const:** Indicates that a variable's value cannot be changed after initialization.
2. **volatile:** Suggests that a variable's value may be altered by external factors (e.g., hardware or other processes). This is often used in embedded systems programming.
3. **static:** Limits the visibility of a variable or function to the file it is defined in. Static variables retain their values between function calls, while static functions can only be called within the same file.
4. **extern:** Specifies that a variable or function is defined in another file, allowing access from the current file.
5. **signed and unsigned:** Used with integer types (like int, short, long) to indicate whether a variable can take both positive and negative values (signed) or only non-negative values (unsigned).
6. **register:** Suggests that a variable should be stored in a processor register for faster access, though modern compilers typically optimize variable storage automatically.

These modifiers can be combined to create more specialized data types, which is essential for writing efficient and correct code, especially in low-level or embedded systems programming.

### Identifiers in C

Identifiers are names assigned to various program elements, including variables, functions, constants, labels, and user-defined data types. Here are the key rules and conventions for using identifiers in C:

1. **Valid Characters:** Identifiers can include letters (uppercase and lowercase), digits, and underscores but must start with a letter or an underscore.
  2. **Length:** Identifiers can be any length, but only the first 31 characters are significant.
  3. **Case Sensitivity:** C is case-sensitive, meaning count and Count are considered different identifiers.
  4. **Reserved Words:** Identifiers cannot match reserved keywords in C (e.g., if, for, while, return).
  5. **Meaningful Names:** Choose meaningful names that reflect the purpose of the identifier. For instance, a variable for a person's age could be named age.
  6. **CamelCase Convention:** For multi-word identifiers, it's common to use CamelCase, capitalizing the first letter of each word except the first (e.g., firstName, lastName).
  7. **Avoid Abbreviations:** Refrain from using abbreviations, as they can be unclear. Instead, opt for descriptive names.
- By adhering to these rules, you can create clear and meaningful identifiers in your C programs.

### Keywords in C

Keywords are reserved words in C that have specific meanings and functionalities. These cannot be used as identifiers (like variable or function names) because they are predefined in the language. Here are the keywords in C:

- auto, double, int, struct
- break, else, long, switch
- case, enum, register, typedef
- char, extern, return, union
- const, float, short, unsigned
- continue, for, signed, void

- default, goto, sizeof, volatile
- do, if, static, while

Some keywords, like int, float, and char, define data types, while others (e.g., if, else, switch) control program flow. The keyword void indicates a function that does not return a value, and typedef is used to create custom data types.

While keywords cannot be used as standalone identifiers, they can be part of larger identifiers (e.g., firstName is valid despite containing the keyword int). Misusing keywords can lead to compilation errors, so it's crucial to avoid using them as identifiers in your programs.

### Symbolic Constants in C

Symbolic constants, or named constants, are identifiers representing values that cannot be changed during program execution. They are declared using the #define preprocessor directive. Here's an example:

```
#define PI 3.14159
int main(){
    float radius = 5.0;
    float circumference = 2* PI * radius;
    return 0;
}
```

In this example, #define creates the symbolic constant PI with a value of 3.14159, used to calculate the circumference of a circle.

#### Advantages of Symbolic Constants:

1. **Readability:** They enhance code readability, especially for frequently used values.
  2. **Maintainability:** Changing a value requires only updating the symbolic constant definition, rather than multiple code locations.
  3. **Type Safety:** Symbolic constants can be defined with specific data types, helping to prevent type errors.
  4. **Debugging:** Debugging is simplified since the value is defined once, making issue identification easier.
- Using symbolic constants contributes to more readable, maintainable, and type-safe code.

### Storage Classes in C

Storage class in C defines where a variable is stored in memory and its visibility within a program. C has four storage classes, each with its own scope and lifetime:

1. **Automatic Storage Class:**
  - Variables declared within a function or block without a specifier have automatic storage.
  - They are created and initialized on each function call and destroyed when the function exits.
  - The keyword auto can be used, but it's optional.
2. **Register Storage Class:**
  - Variables declared with the register specifier are stored in the processor's register for faster access.
  - However, there is a limit to the number of register variables, and the compiler may not always use this storage.
3. **Static Storage Class:**
  - Static variables retain their values throughout the program's execution and are initialized only once at startup.
  - Static variables inside a function are visible only within that function, while those outside have global visibility.
4. **External Storage Class:**
  - Variables declared with the extern specifier are defined outside the current file and accessible from other files.
  - They must be defined elsewhere in the program and are not initialized when declared.

Using appropriate storage classes allows programmers to control variable memory allocation, lifetime, and visibility, ensuring optimal performance and reducing unexpected behavior.

### Enumerations in C

Enumerations in C allow you to define a new data type with a set of named integer constants, making it easier to work with related constants. The syntax for defining an enumeration is as follows:

```
enum enum_name{const1 = value1,
const2 = value2,
const3 = value3}
```

```
const = values,  
...  
};
```

Here, `enum_name` is the name of the enumeration, and each `const` represents a named constant. If no values are specified, constants are assigned sequential integer values starting from 0.

#### Example:

```
#include <stdio.h>  
enum weekday{Monday, Tuesday, Wednesday, Thursday, Friday, Saturday, Sunday  
};  
int main(){  
    enum weekday today=Tuesday;  
    printf("Today is day %d\n", today);  
    return 0;  
}
```

In this example, the enumeration `weekday` defines seven constants for the days of the week. The variable `today` is initialized to `Tuesday`, and its integer value is printed.

Using enumerations improves code readability by giving descriptive names to constants, making it easier to understand their meanings.

#### Command Line Parameters in C

Command line parameters are arguments passed to a C program when executed from the command line. These parameters can modify the program's behavior or provide input data. They are stored in the `argv` array, while the count of parameters is stored in `argc`.

##### Syntax:

To pass command line parameters to a C program, use the following command:

```
$ ./program arg1 arg2 arg3 ...
```

In C, the `main` function can accept these two arguments:

```
int main(int argc, char*argv[]){  
    // program code  
    return 0;  
}
```

- `argc` is the number of command line parameters.
- `argv` is an array of strings containing the parameter values. The first element (`argv[0]`) is the program name.

##### Accessing Command Line Parameters:

You can access individual parameters using array indexing:

```
int main(int argc, char*argv[]){  
    for(int i = 1; i < argc; i++) {  
        printf("Parameter %d: %s\n", i, argv[i]);  
    }  
    return 0;  
}
```

In this example, a loop iterates through the command line parameters starting from index 1, printing each parameter with its index. This provides a way to handle user input directly from the command line, enhancing program flexibility.

### Macros in C

Macros in C allow you to define constants or functions for use throughout your program. They are preprocessor directives defined with the `#define` directive and are processed before compilation.

#### Syntax:

`#define macro_name(value) expression`

- **macro\_name:** The name of the macro.
- **value:** An optional parameter used in the expression.
- **expression:** The code or value that replaces the macro.

#### Example:

To define a constant for Pi:

```
#define PI 3.14159
```

To define a macro for calculating the square of a number:

```
#define SQUARE(x) ((x) * (x))
```

When `SQUARE` is used in the code, `x` is replaced by the provided argument.

#### Caution:

Macros can be powerful but may lead to debugging challenges and unexpected results. It's essential to thoroughly test and debug code that utilizes macros to ensure correct functionality.

### The C Preprocessor

The C preprocessor is a part of the C compiler that performs text manipulation before the compilation process. It handles directives that begin with the `#` symbol and serves several functions:

1. **Macro Expansion:** Replaces macros with their defined values.
2. **Conditional Compilation:** Includes or excludes code based on conditions using directives like `#ifdef`, `#ifndef`, `#if`, and `#else`.
3. **File Inclusion:** Includes other files in the source code using `#include`, which is useful for adding header files.
4. **Line Control:** Manages line numbering in the source code with the `#line` directive, aiding debugging and error message generation.
5. **Compiler-Specific Directives:** Includes directives like `#pragma` that are specific to certain compilers.

The preprocessor operates on the source code before it is compiled, passing the modified code to the compiler for actual compilation. You can invoke the preprocessor separately using the `cpp` command to generate preprocessed code from the source input.

### Break Statement in C

The **break** statement in C is used to terminate the execution of a loop or a switch statement. When a break is encountered, control transfers to the statement following the loop or switch.

#### Syntax:

```
break;
```

#### Usage in Loops:

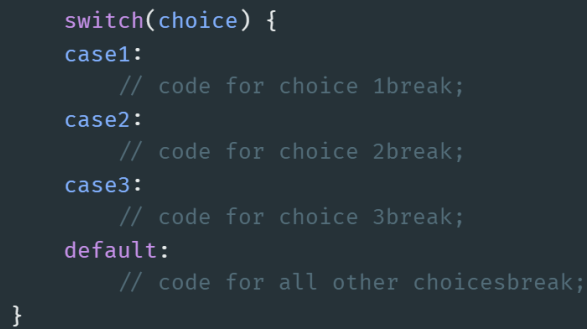
In loops, the `break` statement is often used within an `if` condition. For example:

```
for(int i = 0; i < 10; i++) {  
    if(i == 5) {  
        break;  
    }  
    printf("%d\n", i);  
}
```

In this example, the loop exits when `i` equals 5.

#### Usage in Switch Statements:

In switch statements, `break` terminates the execution of the current case:



```

switch(choice) {
case1:
    // code for choice 1break;
case2:
    // code for choice 2break;
case3:
    // code for choice 3break;
default:
    // code for all other choicesbreak;
}

```

Without break, execution would continue into the next case.

### Continue Statement in C

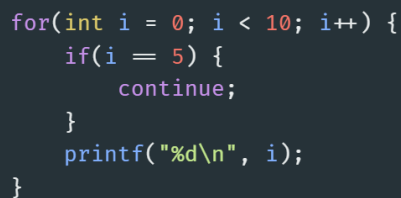
The **continue** statement in C skips the current iteration of a loop and proceeds to the next iteration. When continue is encountered, control jumps to the loop's next iteration, bypassing any remaining statements in the current iteration.

#### Syntax:

**continue;**

#### Example:

In this example, the continue statement skips the iteration when i equals 5:



```

for(int i = 0; i < 10; i++) {
    if(i == 5) {
        continue;
    }
    printf("%d\n", i);
}

```

Here, when i is 5, the continue statement is executed, and the print statement is skipped for that iteration.

Both break and continue are powerful control flow tools that help manage loop execution efficiently, allowing for more concise and readable code.

### Comma Operator in C

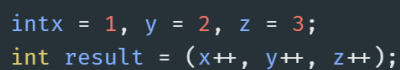
The **comma operator** in C is a binary operator that evaluates two expressions and returns the value of the second expression.

#### Syntax:

expression1, expression2

When using the comma operator, expressions are evaluated from left to right. The result of the first expression is discarded, and the value of the second expression is returned.

#### Example:



```

int x = 1, y = 2, z = 3;
int result = (x++, y++, z++);

```

In this example:

- x++ increments x to 2.
- y++ increments y to 3.

- `z++` increments `z` to 4.

The value of `result` will be 4, the final value of `z`. While the comma operator can simplify complex expressions, it can also reduce code readability, so it should be used with caution.

### Goto Statement in C

The **goto** statement is a jump statement that allows you to transfer control to a labeled statement within the same function.

#### Syntax:

go to label;

Here, label is a user-defined identifier followed by a colon that marks the position to which control is transferred.

#### Example:



```
#include <stdio.h>
int main(){
    int i = 1;
loop:
    printf("%d\n", i);
    i++;
    if(i < 10) {gotoloop;}
    return 0;
}
```

In this code:

- The label `loop` marks the beginning of a simple loop.
- The `goto loop` statement transfers control back to this label, causing the loop to continue until `i` reaches 10.

#### Caution:

While `goto` can be useful for implementing complex logic, it can make code harder to read and debug. It's generally recommended to use structured control flow statements (like `if`, `while`, and `for`) instead of `goto` whenever possible.

### Difference Between Built-in and User-defined Functions in C

In C programming, functions are categorized into **built-in functions** and **user-defined functions**.

#### 1. Built-in Functions:

- **Definition:** Functions that are part of the C standard library and are readily available for use in any C program.
- **Usage:** Common tasks like input/output operations, memory management, and mathematical calculations.
- **Examples:** `printf()`, `scanf()`, `strlen()`, `malloc()`.
- **Performance:** Typically optimized for performance and designed to work efficiently with the underlying system.

#### 2. User-defined Functions:

- **Definition:** Functions created by the programmer to meet specific needs of a program.
- **Characteristics:** Must be explicitly defined and declared in the program.
- **Flexibility:** Can perform any task that can be expressed in code and can be tailored for specific functionality.
- **Examples:** A function to calculate the factorial of a number or to sort an array.

#### Key Differences:

- **Availability:** Built-in functions require no additional code to use, while user-defined functions must be written and declared by the programmer.
  - **Optimization:** Built-in functions are generally more optimized than user-defined ones, which may vary in performance depending on implementation.
  - **Use Cases:** Built-in functions handle common tasks; user-defined functions cater to unique program requirements.
- Both types of functions are essential in C programming, and their use depends on the specific needs of the application.

### Difference Between Call by Value and Call by Reference in C

In C, function arguments can be passed to functions in two primary ways: **call by value** and **call by reference**.

#### 1. Call by Value:

- **Definition:** A copy of the argument value is passed to the function.
- **Effect:** Changes made to the parameter within the function do not affect the original variable.
- **Usage:** Default method for passing arguments in C.

#### 2. Call by Reference:

- **Definition:** The memory address (or reference) of the argument is passed to the function.

- **Effect:** Changes made to the parameter within the function will affect the original variable.
- **Usage:** Used when the function needs to modify the original value.

**Example:**

```
#include<stdio.h>voidchange_value(inta, int*b){
    a = a + 10;    // Changes 'a' (call by value)*b = *b +
10; // Changes 'b' (call by reference)}
intmain(){
    intx = 10, y = 10;
    printf("Before function call: x = %d, y = %d\n", x,
y);
    change_value(x, &y);
    printf("After function call: x = %d, y = %d\n", x, y);
    return0;
}
```

**Output:**

Before function call :x =10,y =10

After function call :x =10,y =20

**Summary:**

- **Call by Value:** Passes a copy; original variable remains unchanged.
- **Call by Reference:** Passes a reference (pointer); original variable can be modified.

### Recursive Functions in C

A **recursive function** is a function that calls itself either directly or indirectly.

This approach can be an elegant solution for problems that may be complex when using iterative methods.

**Key Characteristics:**

- **Base Case:** Every recursive function must have a condition under which it stops calling itself, known as the base case.
- **Recursive Case:** The part of the function where it calls itself to work towards the base case.

**Example:**

```
#include<stdio.h>
intfactorial(int n){
    if(n == 0) // Base casereturn1;
    else// Recursive casereturnn * factorial(n - 1);
}

int main(){
    int num = 5;
    printf("Factorial of %d is %d\n", num,factorial(num));
    return 0;
}
```

**Output:**

Factorial of 5 is 120.

**Advantages of Recursion:**

- Simplifies code for problems like tree traversal, factorial calculation, etc.
- Can lead to more straightforward and readable code.

**Considerations:**

- Recursive functions may lead to stack overflow if the recursion depth is too high.
- Performance may be less efficient compared to iterative solutions due to function call overhead.



## Arrays in C

An **array** is a collection of elements of the same type, stored in contiguous memory locations. Arrays can be classified into two main types: **linear arrays** and **multidimensional arrays**.

### Linear Arrays

A linear array, also known as a one-dimensional array, consists of a single row of elements. It can hold multiple values of the same data type, indexed from 0 to n-1 (where n is the size of the array).

#### Declaration and Initialization Example:

In this example:

- myArray is declared as an array that can hold 5 integers.
- It is initialized with the values 1, 2, 3, 4, and 5.

### Multidimensional Arrays

A multidimensional array is an array with more than one dimension. The most common type is a two-dimensional (2D) array, which can be thought of as a table or matrix.

#### Declaration and Initialization Example:

```
int myArray[3][4]={
    {1, 2, 3, 4}
    ,{5, 6, 7, 8}
    ,{9, 10, 11, 12}
};
```

In this example:

- myArray is declared as a 2D array with 3 rows and 4 columns.
- Each element can be accessed using two indices: the first for the row and the second for the column.

#### Key Characteristics

- **Linear Arrays:**
  - Only one dimension.
  - Accessed using a single index.
  - Can be visualized as a single row of values.
- **Multidimensional Arrays:**
  - Multiple dimensions (e.g., 2D, 3D).
  - Accessed using multiple indices.
  - Can be visualized as tables or matrices.

#### Memory Storage

- Both linear and multidimensional arrays are stored in memory in a contiguous block.
- Multidimensional arrays, particularly 2D arrays, are stored in **row-major order**, meaning that all elements of a row are stored consecutively in memory. This can impact performance when accessing elements, especially in large arrays.

#### Accessing Array Elements

You can access elements in an array using indices:

- For a linear array: myArray[i]
- For a 2D array: myArray[row][column]

#### Example of Accessing Elements

```
#include<stdio.h>int main(){
    int myArray[3][4] = {
        {1, 2, 3, 4},
        {5, 6, 7, 8},
        {9, 10, 11, 12}
    };
    printf("Element at row 1, column 2: %d\n",
```

```
myArray[1][2]); // Output: 7return0;
}
```

## Conclusion

Arrays in C are a fundamental data structure that allows for efficient storage and manipulation of collections of data. Understanding linear and multidimensional arrays is essential for effective programming in C, as they form the basis for more complex data structures and algorithms.

## Passing Arrays to Functions in C

In C programming, arrays can be passed to functions as arguments. When an array is passed to a function, a pointer to the first element of the array is actually passed. This allows the function to access and manipulate the original array.

### Example of Passing an Array to a Function

Here's an example that demonstrates how to pass an array to a function:

```
#include<stdio.h>voidprintArray(intarr
[], intsize){
    for(inti = 0; i < size; i++) {
        printf("%d ", arr[i]);
    }
    printf("\n");
}

intmain(){
    intmyArray[] = {1, 2, 3, 4, 5};
    intsize = sizeof(myArray) /
sizeof(myArray[0]);
    printArray(myArray, size);
    return0;
}
```

### Explanation:

- The function `printArray` takes an integer array and its size as parameters.
- Inside the function, a loop iterates through the array and prints each element.
- In `main`, we declare and initialize `myArray` and calculate its size using `sizeof`.
- We then pass both the array and its size to `printArray`.

## Strings in C

In C, a **string** is a sequence of characters stored in an array of characters. Strings are typically null-terminated, meaning they end with a null character (`'\0'`), which indicates the end of the string.

### Example of a String

```
char myString[] = "Hello, world!";
```

In this example:

- `myString` is an array of characters containing the string "Hello, world!".
- The size of the array is automatically determined, including space for the null terminator.

## Difference Between Arrays and Strings

- **Arrays:** Can hold any type of data (e.g., integers, floats, characters).
- **Strings:** A specific type of array that holds only characters and must be null-terminated.

### Memory Allocation in C

C uses two primary memory models for storing data: the **stack** and the **heap**.

#### Stack Memory

- Local variables are stored on the stack, which follows a Last-In-First-Out (LIFO) structure.
- The size of the stack is limited and determined at compile-time.

#### Heap Memory

- Memory can be dynamically allocated on the heap using functions like `malloc()` and `free()`.
- Heap memory is not organized in a specific order and can grow as needed, limited by available system memory.

### Parameterized and Non-Parameterized Macros

Macros in C are defined using the `#define` preprocessor directive and can be categorized as parameterized or non-parameterized.

#### Non-Parameterized Macros

Defined without any arguments:

Code

```
#define Pi 3.14159
```

#### Parameterized Macros

Defined with arguments:

Code

```
#define SQUARE(x) ((x) * (x))
```

- Parameterized macros can simplify code but can also complicate debugging.

### Dynamic Memory Allocation

Dynamic memory allocation allows you to allocate memory at runtime. The functions used for dynamic memory management in C include `malloc()`, `calloc()`, `realloc()`, and `free()`.

#### Example of Dynamic Memory Allocation

```
#include<stdio.h>#include<stdlib.h>intmain(){
    intn;
    int* ptr;
    printf("Enter the number of integers to allocate:
"); scanf("%d", &n);

    ptr = (int*)malloc(n * sizeof(int));
    if(ptr == NULL) {
        printf("Error: Unable to allocate memory\n");
        exit(1);
    }
    for(inti = 0; i < n; i++) {
        printf("Enter an integer: ");
        scanf("%d", &ptr[i]);
    }
}
```

```
        scanf("%d", &ptr[i]);
    }
    printf("The integers you entered are: ");
    for(int i = 0; i < n; i++) {
        printf("%d ", ptr[i]);
    }
    free(ptr);
    return 0;
}
```

**Explanation:**

- The program prompts the user for the number of integers to allocate.
- It uses malloc() to allocate memory dynamically.
- If allocation fails, it prints an error message and exits.
- After using the memory, it frees it using free() to prevent memory leaks.

**Conclusion**

Understanding how to pass arrays to functions, handle strings, allocate memory dynamically, and use macros are essential skills in C programming. These concepts provide the foundation for effective memory management and code organization in C.